

# LD11 Default Automatic Density Backend Policy

Physical-resolution-first dense versus local-sparse selection

mdstats development specification

2026-07-21

## Contents

|          |                                         |          |
|----------|-----------------------------------------|----------|
| <b>1</b> | <b>Status and scope</b>                 | <b>1</b> |
| <b>2</b> | <b>Motivation</b>                       | <b>1</b> |
| <b>3</b> | <b>Public defaults</b>                  | <b>2</b> |
| <b>4</b> | <b>Normative default planning order</b> | <b>2</b> |
| 4.1      | Resolve scientific resolution . . . . . | 2        |
| 4.2      | Estimate both realizations . . . . .    | 2        |
| 4.3      | Select transactionally . . . . .        | 3        |
| <b>5</b> | <b>No silent broadening rule</b>        | <b>3</b> |
| <b>6</b> | <b>Selection metadata</b>               | <b>3</b> |
| <b>7</b> | <b>Compatibility and migration</b>      | <b>3</b> |
| <b>8</b> | <b>Validation requirements</b>          | <b>4</b> |
| <b>9</b> | <b>Algorithm attribution</b>            | <b>4</b> |

## 1 Status and scope

This specification governs **LD11**, implemented in `mdstats 0.19.65a0`. It changes the production defaults for atomic occupancy, framework-vertex occupancy, and framework-edge arc-length density fields from the historical legacy-spectral dense path to the canonical periodized operator with automatic backend selection.

LD11 changes execution-policy defaults only. It does not change the density measure, frame weighting, periodic registration, adaptive broadening target, normalization, highest-density-region semantics, or mesh rendering.

## 2 Motivation

The historical defaults resolved adaptive grid refinement under a dense logical voxel allowance before choosing a storage backend. For localized fields, a dense field can be infeasible while a block-sparse realization of the same logical grid is inexpensive. Applying the dense allowance first can therefore enlarge the grid interval and, through

$$\sigma = rh,$$

artificially enlarge the Gaussian bandwidth even though the requested physical resolution is feasible with sparse storage.

The production default must instead choose the implementation for the requested scientific field. It must not choose the scientific field to suit one implementation.

### 3 Public defaults

The shared option records become:

```
DensityKernelOptions(
    smoothing_operator="discrete_periodized_v1",
    kernel_tail_tolerance=1.0e-8,
)
```

```
DensityStorageOptions(
    grid_backend="auto",
    local_block_shape=(16, 16, 16),
    sparse_activation_fraction=0.20,
)
```

Consequently, these ordinary constructions require no backend or operator arguments:

```
AtomicDensityOptions()
FrameworkDensityOptions()
```

Explicit overrides remain supported:

```
# Reproducible forced dense canonical field.
DensityStorageOptions(grid_backend="dense")
```

```
# Reproducible forced local-sparse canonical field.
DensityStorageOptions(grid_backend="local_sparse")
```

```
# Historical estimator compatibility; dense-only.
DensityKernelOptions(smoothing_operator="legacy_spectral_v1")
DensityStorageOptions(grid_backend="dense")
```

`legacy_spectral_v1` combined with `auto` or `local_sparse` remains rejected because no scientifically identical sparse implementation exists.

## 4 Normative default planning order

For every automatic field, the implementation must perform these steps in order.

### 4.1 Resolve scientific resolution

Resolve the nominal grid from the display-cell vector lengths and `grid_interval`, then apply spread-aware refinement. Automatic and explicit local-sparse modes resolve this step independently of the dense voxel allowance. The result is one immutable pair

$$((N_1, N_2, N_3), \sigma).$$

The dense and sparse candidates must receive exactly this same grid shape and bandwidth.

### 4.2 Estimate both realizations

Before allocating a scalar field, construct bounded candidate estimates for:

- dense CIC plus canonical periodized convolution;
- local-sparse CIC, finite-support target planning, block packing, and bounded realization.

Each estimate records feasibility, peak package-owned memory, work, active-node fraction, retained storage, and the limiting reason when infeasible.

### 4.3 Select transactionally

Apply the implemented LD4 field-local policy, then evaluate all feasible whole-scene backend combinations under the active runtime-derived LD10 memory, thread, and wall-time budget. The chosen combination must be approved before scientific field allocation.

Broad fields may select dense. Localized fields may select local-sparse. The choice is field-specific; a multi-species scene may use both backends.

## 5 No silent broadening rule

In default automatic mode, a dense voxel limit must not reduce the logical grid, increase the grid interval, or increase the Gaussian bandwidth.

If dense is infeasible but sparse is feasible at the requested resolution, sparse must be selected.

If neither realization is feasible, preparation must fail with `GraphComplexityError` and report both candidate reasons. It must not silently coarsen the field. The user may then change a scientific resolution option or increase the explicit runtime budget knowingly.

An explicit forced-dense request is different: it requests a dense realization under the dense budget. Existing dense adaptive-budget behavior remains available for that expert/reproducibility path and is recorded as budget-limited metadata.

## 6 Selection metadata

Every automatically prepared field must record:

```
requested_storage_backend = auto
storage_backend = dense | local_sparse
backend_selection.policy
backend_selection.reason
backend_selection.dense
backend_selection.local_sparse
logical_node_count
grid_shape
gaussian_bandwidth
sample_sd_reference
adaptive_smearing_triggered
adaptive_smearing_budget_limited
```

For the default automatic path, `adaptive_smearing_budget_limited=True` may not be caused solely by the dense voxel allowance. The resolved field and both backend candidates must be auditable from metadata and the scene planning record.

## 7 Compatibility and migration

This is a deliberate alpha-version default migration.

- Existing serialized option records remain explicit and retain their stored operator and backend.
- Explicit dense, local-sparse, canonical, and legacy modes retain their meanings.
- Code that consumes prepared fields must use the `ScalarField3D` and `PeriodicNodeFieldAccess` interfaces. It must not assume that a default field owns a dense `.values` array.
- `to_dense_values(max_nodes=...)` remains an explicit bounded debugging and comparison operation, not a default rendering path.

## 8 Validation requirements

LD11 requires focused tests for:

1. `AtomicDensityOptions()` and `FrameworkDensityOptions()` resolving to `discrete_periodized_v1` plus `grid_backend="auto"`;
2. a localized field selecting sparse without explicit backend arguments;
3. a broad field selecting dense automatically;
4. a dense voxel allowance smaller than the logical grid not reducing automatic resolution when sparse is feasible;
5. adaptive refinement reaching the configured  $\sigma/s$  target under the same condition;
6. explicit legacy-spectral dense compatibility;
7. explicit invalid legacy-spectral automatic selection being rejected;
8. complete backend-selection provenance and canonical JSON round trips.

## 9 Algorithm attribution

The density deposition, periodized Gaussian operator, and HDR construction retain their existing cited sources. The LD11 planning order and automatic default migration are project-specific engineering policy, not an adaptation of an external backend-selection algorithm.